Ex.No: 1 **IMPLEMENT LINEAR SEARCH**

Date:

AIM:

To implement Linear search and determine the time required to search an element and plot a graph of time Taken versus n.

ALGORITHM:

Step 1: Start.

Step 2: Define linear search function.

Step 3: Define run-experiment to find the time taken.

Step 4: Create an array with number of elements in the list to find the time taken.

Step 5: Plot the time taken vs n using pyplot.

Step 6: Stop.

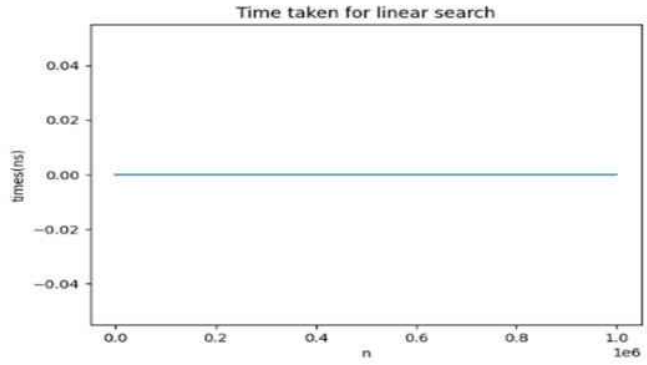
PROGRAM:

```
import time
import matplotlib.pyplot as plt
def linear_search(arr,x):
    for i in range(len(arr)):
        if arr[i]==x:
            return i
    return-1
def  run_experiment(n):
```

```
    arr=list(range(n))
        x=n-1
        start=time.time()
        result=linear_search(arr,n)
        end=time.time()
        print(n,":",end-start,"is the time taken")
        return(end-start)
ns=[10,100,1000,10000,100000,1000000]
times=[run_experiment (n)for n in ns]
plt.plot(ns,times)
plt.xlabel("n")
plt.ylabel("times(n)")
plt.title("Time taken for linear search")
plt.show()
```

OUTPUT:

```
10 : 0.0 is the taken time
100 : 0.0 is the taken time
1000 : 0.0 is the taken time
10000 : 0.0 is the taken time
100000 : 0.0 is the taken time
1000000 : 0.0 is the taken time
```



Result:

Thus the program has been executed successfully

Ex.No:2 **IMPLEMENT RECURSIVE BINARY SEARCH**

Date:

AIM:

To implement recursive binary search and determine the time required to search an element and plot a graph of time Taken versus n.

ALGORITHM:

Step1: Start.

Step 2: Define recursive binary search.

Step 3: Import time module to determine the time taken to search an element.

Step 4: Plot the time taken vs n using pyplot.

Step 5: Stop.

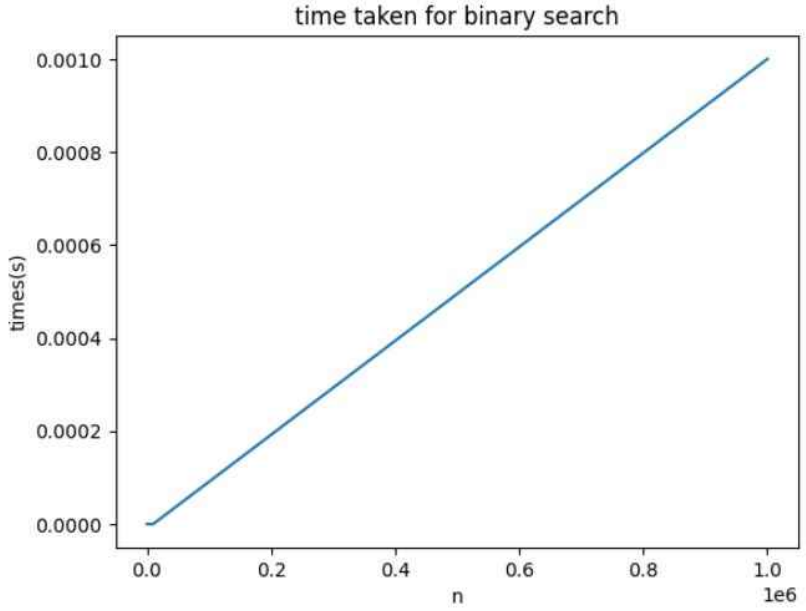
PROGRAM:

```
import time
import matplotlib.pyplot as plt
def binary_search(arr,low,high,x):
if low<=high:
                mid=(low+high)//2
                if (arr[mid]==x):
return mid
elifarr[mid]<x:
                    return binary_search(arr,mid+1,high,x)
                else:
                    return binary_search(arr,low,mid-1,x)
```

```
                else:
                    return -1
def  run_experiment(n):
arr=list(range(n))
            x=n-1
start_time=time.time()
            result=binary_search(arr,0,n-1,x)
end_time=time.time()
print(n,":",end_time-start_time,"is the time taken")
            return (end_time-start_time)
        ns=[10,100,1000,10000,100000]
        times=[run_experiment(n) for n in ns]
plt.plot(ns,times)
plt.xlabel("n")
plt.ylabel("times(s)")
plt.title("time taken for binary search")
plt.show()
```

OUTPUT:

```
10 : 0.0 is the time taken
100 : 0.0 is the time taken
1000 : 0.0 is the time taken
10000 : 0.0 is the time taken
1000000 : 0.0009999275207519531 is the time taken
```



RESULT:

Thus the implementation of binary search has been executed successfully.